

Lesson: Tuples and Dictionaries in Python

Introduction

In previous lessons, you learned how to use **lists** to store multiple values. Python also provides other powerful data structures:

- **Tuples** → ordered, *unchangeable* collections
- **Dictionaries** → collections of *key-value pairs*

These structures are used heavily in real-world programming for organizing and managing data efficiently.

In this lesson, you will learn:

- how tuples work and when to use them
- how dictionaries store and retrieve data
- how both structures are used in real applications

Part 1: Tuples in Python

What Is a Tuple?

A **tuple** is similar to a list, but it is **immutable**, meaning it cannot be changed after it is created.



```
coordinates = (10, 20)
```

Key Characteristics:

- Ordered
- Cannot be modified (immutable)
- Uses parentheses `()`

Accessing Tuple Elements

Like lists, tuples use indexes starting at 0.



```
point = (5, 10, 15)
print(point[0])    # 5
print(point[1])    # 10
```

Why Use Tuples?

Use tuples when:

- data should not change
- you want to protect values from modification
- working with fixed data (coordinates, settings, etc.)

Example: Storing Coordinates



```
location = (40.7128, -74.0060)
print(Latitude:, location[0])
print(Longitude:, location[1])
```

Tuple Packing and Unpacking

Python allows you to assign multiple values at once.



```
person = (Alice, 25)
name, age = person
print(name)
print(age)
```

Attempting to Modify a Tuple (Error)



```
numbers = (1, 2, 3)  
numbers[0] = 10 # ERROR
```

This will cause an error because tuples cannot be changed.

Part 2:

Dictionaries in

Python

What Is a Dictionary?

A **dictionary** stores data as **key-value pairs**.



```
student = {  
    name: Alice,  
    age: 20,  
    major: Computer Science  
}
```

Key Characteristics:

- Uses `{ }` (curly braces)
- Each item has a key and a value
- Keys must be unique

Accessing Dictionary Values



```
print(student[name])    # Alice  
print(student[age])     # 20
```

Adding or Updating Values



```
student[age] = 21          # update  
student[gpa] = 3.8        # add new key  
print(student)
```

Removing Items



```
student.pop(major)  
print(student)
```

Looping Through a Dictionary

Loop through keys:



```
for key in student:  
    print(key)
```

Loop through values:

```
for value in student.values():  
    print(value)
```

Loop through key-value pairs:

```
for key, value in student.items():  
    print(key, :, value)
```

Real-World Use Cases

1. Student Record System



```
student = {  
    name: Bob,  
    age: 22,  
    grade: A  
}  
print(f{student['name']} earned a grade of {student['grade']})
```

2. Simple Login System



```
users = {  
    admin: 1234,  
    user1: abcd  
}  
username = input(Enter username: )  
password = input(Enter password: )  
if username in users and users[username] == password:  
    print(Login successful)  
else:  
    print(Login failed)
```

3. Counting Items



```
word = apple
count = {}
for letter in word:
    if letter in count:
        count[letter] += 1
    else:
        count[letter] = 1
print(count)
```

4. Storing Product Information



```
product = {  
    name: Laptop,  
    price: 999.99,  
    in_stock: True  
}  
if product[in_stock]:  
    print(Product is available)
```

Tuples vs Lists vs Dictionaries

Feature	List	Tuple	Dictionary
Syntax	[]	()	{ }
Ordered	Yes	Yes	Yes (Python 3.7+)
Mutable	Yes	No	Yes
Use Case	Collections	Fixed data	Key-value data

Combining Tuples and Dictionaries

You can use tuples inside dictionaries:



```
locations = {  
    NYC: (40.7128, -74.0060),  
    LA: (34.0522, -118.2437)  
}  
print(locations[NYC])
```

Common Beginner Mistakes

 Using wrong key

```
print(student[Name]) # ERROR (case-sensitive)
```

 Correct:

```
print(student[name])
```

 Forgetting keys must exist

```
print(student[gpa]) # ERROR if not defined
```

 Safe version:

```
print(student.get(gpa, Not found))
```

Full Program Example



```
# Tuple example
person = (Alice, 30)
name, age = person
print(Name:, name)
print(Age:, age)
# Dictionary example
user = {
    username: admin,
    password: 1234
}
entered_user = input(Enter username: )
entered_pass = input(Enter password: )
if entered_user in user and user[password] == entered_pass:
    print(Access granted)
else:
    print(Access denied)
```

Why Tuples and Dictionaries Matter

These data structures are used everywhere in programming:

- Tuples → coordinates, fixed settings, return values
- Dictionaries → APIs, databases, configuration, user data

They allow you to organize data in ways that lists alone cannot.

Summary

Tuples:

- Store ordered data that cannot be changed
- Use parentheses `()`
- Support indexing and unpacking

Dictionaries:

- Store key-value pairs using `{}`
- Access values using keys
- Add, update, and remove data
- Loop through keys and values

Together with lists, tuples and dictionaries form the **core data structures of Python** and are essential for building real-world applications.